posco

설비 고장을 예측하는 AI 데이터 분석

예외처리 및 종합 실습

데이터 처리 파이프라인

posco

CONTENTS

데이터 처리 파이프라인

PART 1

파이프라인 설계

종합 실습 개요 · 파이프라인 설계

PART 2

읽기와 분류

데이터 읽기 · 조건 분류

PART 3

통계와 방어

통계 함수 · 불량 데이터 방어

PART 4

리포트 작성

리포트 구성 · 파일 저장

종합 실습 개요 — 무엇을 만드는가

배운 문법을 모아 하나의 분석 프로그램을 완성 — 새 문법은 없음
앞 단계 결과가 뒤 단계의 재료가 되는 연결 구조 = 파이프라인

변수 · 조건문 · 반복문 · 함수 · 파일 입출력 · 예외처리

ICT 검사 CSV를 읽어 부품별로 정리하고, 불량을 걸러내고, 통계를 내어 리포트로 저장

만들 프로그램

데이터 분석 실무의 가장 기본적인 흐름이다.

새 문법은 없다

한 단계씩 쌓아 올리므로 막혀도 어느 단계인지 추적 가능

파이프라인 단계별 흐름

읽기 → 분류 → 계산 → 방어 → 저장. 각 단계는 앞 단계 결과를 이어받는다.



CSV 21행 → rows 21 → 부품 5종(제외 2) → 측정값 17건 → 유효 16 → 리포트

줄어드는 지점마다 이유를 말할 수 있어야 파이프라인을 이해한 것

PRACTICE

실습 1. CSV 읽기

목표

CSV를 안전하게 열어 헤더와 데이터 행 분리하기

단계

① 분석할 CSV 파일의 경로를 지정

② 파일을 열어 첫 줄은 헤더로, 나머지는 데이터 행으로 분리

③ 헤더와 데이터 행을 함께 돌려주는 함수로 구성

④ 만약 파일이 없다면 안내문을 출력하고 빈 헤더·빈 데이터 행을 반환

⑤ 데이터가 몇 행인지 출력해 확인

예상 결과

헤더 7개 항목과 데이터 21행이 분리되어 출력

실습 1. CSV 읽기 정답

헤더와 데이터를 분리해 담는다 — `next()` 로 한 줄, 나머지는 `list()`

CODE · 정답

```
with open(name, encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)   # 첫 줄 = 헤더
    rows = list(reader)     # 나머지 = 데이터
```

EXPLAIN

next(reader)
첫 줄 = 헤더

list(reader)
나머지 = 데이터 행

파일 없음은 함수 안에서 처리
`FileNotFoundError` 까지 처리한다.

CHECKPOINT
헤더 7개 항목, 데이터 21행이 맞는지 확인 — 이 함수가 다음 단계들의 출발점

PRACTICE

실습 2. 부품별로 분류하기

목표

읽어온 줄을 부품명 기준으로 모으기

단계

① 앞 단계에서 읽어온 데이터 행을 한 줄씩 확인

② 각 줄에서 부품명을 꺼내 분류 기준으로 사용

③ 처음 보는 부품명이라면 그 부품의 빈 목록을 먼저 준비

④ 해당 부품의 목록에 그 줄을 추가

⑤ 만약 부품명이 비어 있다면 분류에서 제외하고 제외한 줄 수를 따로 세기

⑥ 부품이 몇 종이고 제외가 몇 줄인지 출력

예상 결과

부품 5종(A2·B2·D2·E2·F2)으로 묶이고 제외 2행이 기록

실습 2. 부품별로 분류하기 정답

처음 보는 부품명이면 빈 목록을 먼저 준비 — 부품명이 빈 행은 제외하고 수를 기록

CODE · 정답

```
p = row[1]                          # 부품명
if p == "":
    dropped += 1                    # 부품명 없는 행은 제외하고 기록
    continue
by_part.setdefault(p, []).append(row)
```

EXPLAIN

if m not in …

처음 보는 부품이면 빈 목록 준비
먼저 만든다. 이 조건이 없으면
`KeyError` 가 발생한다.

부품명이 비면 제외하고 건수 기록
실제 정답은 이 흐름을 함수로 감싼다

CHECKPOINT

부품 5종으로 묶이고 제외 2행이 따로 기록되는지 확인

개념코드에서 두 방법을 직접 비교해 본다.

딕셔너리에 값을 모으는 두 가지 방법

조건문으로 준비하는 방법과 setdefault로 한 줄에 처리하는 방법

CODE

```
if p not in by_part:
    by_part[p] = []
by_part[p].append(row)

# 같은 동작을 한 줄로
by_part.setdefault(p, []).append(row)
```

EXPLAIN

if not in

조건문 방식은 흐름이 눈에 보임

setdefault

setdefault는 키가 없을 때만 기본값을 넣고 그 값을 돌려줌

두 방식의 결과는 완전히 동일
조건을 빠뜨리면 `KeyError` 발생

CHECKPOINT

같은 데이터로 두 방식을 각각 돌려 결과가 같은지 직접 비교

PRACTICE

실습 3. 통계 함수 만들기

목표

숫자 칸의 개수·평균·최대·최소를 구하는 함수 만들기

단계

① 데이터 행 목록과 몇 번째 칸을 볼지를 함께 받는 함수를 정의

② 각 줄에서 지정한 칸의 값을 숫자(float)로 변환

③ 만약 숫자로 변환이 불가능하거나 그 칸이 아예 없다면 그 줄은 건너뛰기

④ 모인 숫자가 하나도 없다면 계산하지 않고 없음을 뜻하는 값을 반환

⑤ 숫자가 있다면 개수·평균·최댓값·최솟값을 계산

⑥ 네 결과를 하나로 묶어 반환

예상 결과

측정값 17건의 통계가 계산되고, 값이 없으면 없음을 뜻하는 값이 반환

실습 3. 통계 함수 만들기 정답

값이 없으면 None 을 반환 — `ZeroDivisionError` 를 막는다.

CODE · 정답

```
if not values:
    return None   # 값이 없으면 None (0 나누기 방지)
mean = sum(values) / len(values)
return mean
```

EXPLAIN

if not values:
값이 없으면 None 반환
`ZeroDivisionError` 를 막아준다.

mean = sum / len
값이 있으면 합을 개수로 나눠 평균.

실제 정답은 **최대 · 최소도**
함께 **딕셔너리로 반환**한다.

CONSOLE

측정값 통계 — 개수 17건 · 평균 1623.22 · 최대 4366.23 · 최소 -999.0

PRACTICE

실습 4. 불량 데이터 걸러내기

목표

형변환 실패와 범위 밖 값을 모두 제외하기

단계

① 측정값의 정상 범위를 최솟값·최댓값으로 정하기

② 각 줄의 측정값을 숫자로 변환

③ 만약 숫자로 변환이 불가능하다면 그 줄은 제외

④ 숫자로 변환됐더라도 정상 범위를 벗어났다면 직접 예외를 일으켜 제외

⑤ 제외한 줄은 몇 번째 줄인지와 제외 사유를 함께 기록

⑥ 남은 정상 줄만 모아 반환

예상 결과

유효 16행만 남고 제외 5건의 줄 번호와 사유가 기록

실습 4. 불량 데이터 걸러내기 정답

범위 밖이면 raise 로 직접 예외를 일으켜 차단한다.

CODE · 정답

```
if v < lo or v > hi:
    raise ValueError("범위 밖")   # 범위 밖이면 직접 차단
```

CONSOLE

유효 16건 · 제외 5건 — 결측 1 · 빈칸 2 · 짧은 줄 1 · 범위 밖 1

범위 검증을 넣어야 -999가 걸러지고 평균이 1787.11로 바로잡힘

EXPLAIN

float() 단계

변환 실패는 자동 예외
자동으로 발생한다.

raise ValueError

범위 밖은 직접 차단
직접 예외를 일으킨다.

두 경우 모두 **같은 except** 에서 잡힌다.

좋은 리포트의 원칙

핵심 먼저, 단위 붙이기, 구분선 활용 — 세 가지 원칙으로 정리한다.

01

핵심 먼저

전체 · 유효 · 제외 건수를
맨 위에 놓는다.

읽는 사람이 가장 먼저
알고 싶은 정보부터.

02

단위 붙이기

숫자마다 **'도' · '개'** 같은
단위와 설명을 함께.

그래야 사람이 바로
의미를 이해할 수 있다.

03

구분선 활용

줄을 나누고 **구분선**으로
영역을 가른다.

리포트 줄들을 **리스트에 모아**
한 번에 파일로 쓴다.

데이터를 다 분석해도 사람이 못 읽으면 소용없음 — 읽는 사람이 가장 알고 싶은 수치부터 배치

PRACTICE

실습 5. 리포트 파일로 저장하기

목표

분석 결과를 사람이 읽을 수 있는 파일로 남기기

단계

① 리포트에 넣을 문장을 한 줄씩 만들어 목록에 모으기

② 맨 위에는 전체 행 수·유효 건수·제외 건수를 요약으로 배치

③ 구분선을 넣은 뒤 그 아래에 평균·최댓값·최솟값을 배치

④ 모든 숫자에 무슨 값인지 알 수 있는 항목명을 함께 표기

⑤ 모아둔 줄을 하나의 텍스트 파일로 저장

⑥ 저장한 파일을 다시 열어 내용이 제대로 들어갔는지 확인

예상 결과

요약과 세부 통계가 담긴 텍스트 파일이 생성

posco

실습 5. 리포트 파일로 저장하기 정답

줄을 모아 한 번에 파일로 쓴다 — `"\n".join(lines)`

CODE · 정답

```
lines.append("=== 리포트 ===")
lines.append(f"평균 {mean}도")
with open(name, "w") as f:
    f.write("\n".join(lines))
```

EXPLAIN

lines.append(...)
줄을 목록에 모아
리스트에 모은다.

f.write(...)
마지막에 한 번에 파일로 쓴다.
`with open` 의 쓰기 모드 `"w"` 사용.

CHECKPOINT
저장한 `report.txt` 를 **메모장이나 VS Code로 직접 열어** 사람이 읽기 좋은지 확인한다.

저장된 리포트 파일 확인

실제로 만들어진 09_report.txt의 내용

CHECK

파일을 열어 직접 눈으로 확인

숫자만 있고 단위가 없으면 다시 물어보게 됨
제외된 행이 몇 건인지 함께 남기는 것이 핵심

분석 결과는 남겨야 근거가 됨
화면 출력은 프로그램이 끝나면 사라짐

09_report.txt

```
=== ICT 검사 분석 리포트 ===
전체 21행 · 유효 16건 · 제외 5건

DIVIDER

측정값 통계

평균 — 1787.11
최대 — 4366.23
최소 — 0.0
```

PRACTICE

실습 6. 부품별 통계 검증하기

목표

부품별 개수의 합이 전체 유효 개수와 맞는지 확인하기

단계

① 앞서 만든 부품별 분류와 통계 함수를 그대로 재사용

② 부품마다 측정값 통계를 각각 계산

③ 부품별 개수를 모두 더해 합계를 구하기

④ 그 합계와 앞 단계에서 남은 전체 유효 건수를 비교

⑤ 두 값이 같다면 통과, 다르다면 어느 단계에서 어긋났는지 확인

예상 결과

부품별 개수의 합 16과 전체 유효 개수 16이 일치

실습 6. 부품별 통계 검증하기 정답

부품별 개수의 합과 전체 유효 개수를 비교

CODE · 정답

```
for m in sorted(by_type):
    s = calc_stats(by_type[m], 2)
    check_sum += s["count"]
print(check_sum == len(valid))   # True면 통과
```

EXPLAIN

for m in sorted(...)
부품별로 통계 함수 재사용
앞에서 만든 함수와 딕셔너리를 그대로 재사용
그대로 가져다 쓴다.

print(check_sum == len(valid))
부품별 합이 전체 유효 개수와
같은지 비교 — **True 면 통과**.

CONSOLE
A2 4 · B2 3 · D2 3 · E2 3 · F2 3 — 합 16, 전체 유효 개수와 일치